
PROJET FIL ROUGE 1

UPSSIBOT

Projet d'intégration des compétences acquises en 3A SRI



Équipe Projet :

Abdelhafidh GHOULEM

Arthus DORIATH

Joan BELUSCA

Victor CHALUMEAUX

Encadrant / Client :

M. Julien VANDERSTRAETEN

Livrable :

Rapport Technique Final

Table des matières

1	Introduction Générale	3
1.1	Contexte du Projet	3
1.2	Architecture du Projet	3
1.3	Objectifs et Répartition	4
2	Pilotage par Commandes (ManualPilot)	5
2.1	Introduction	5
2.1.1	Architecture du ManualPilot	6
2.2	Étapes Détaillées du Pipeline	8
2.2.1	Le mot (Token)	8
2.2.2	Séparation en liste de tokens (cutter)	8
2.2.3	Chargement du vocabulaire dans TreeMap (vocabulary)	9
2.2.4	Analyse des types (analysis)	9
2.2.5	La structure de Commande (command.h)	10
2.2.6	Création de la liste de commandes (Parser)	10
2.2.7	L'Exécution de la liste de commande (executor)	12
2.2.8	Simulation Graphique et Communication (C → Python)	12
3	Configuration (ConfigLoader)	14
3.1	Gestion de la configuration	14
3.1.1	Structure des fichiers de configuration	14
3.1.2	Chargement et utilisation des paramètres	14
3.2	Implémentation du module ConfigLoader	15
3.2.1	Construction du chemin et ouverture du fichier	15
3.2.2	Lecture du fichier et récupération de la clé	15
3.2.3	Nettoyage des chaînes	15
4	Interaction Utilisateur et Modes de Contrôle (ControlMod)	16
4.1	Interface vocale	16
4.1.1	Principe de fonctionnement	16
4.1.2	Exécution du script Python	16
4.2	Interface utilisateur et gestion des menus	16
4.2.1	Principe général	17
4.2.2	Fonctionnalités accessibles	17
4.2.3	Rôle architectural	17
5	Journalisation et Historique des événements (History)	18
5.1	Rôle du module d'historique	18
5.2	Niveaux d'événements	18
5.3	Initialisation et création du fichier	18
5.4	Enregistrement des événements	19

5.5	Fermeture de l'historique	19
6	PARTIE VISION	
	(DORIATH + BELUSCA)	20
6.1	Organisation	20
6.2	image.h	20
6.2.1	line.h	20
6.2.2	color.h	22
6.2.3	détails sur l'isolation des couleurs	23
6.3	vision.h	23
6.3.1	Détection d'objets	23
6.3.2	Reconnaissance de Sphère	23
6.3.3	Résultats	25
6.3.4	Géométrie et réflexion pour le Cube	27
7	UTILITAIRES (DORIATH)	28
7.1	matrix.h	28
7.2	treeMap.h	28
7.3	Vérification qualité	28
8	Bilan	30
8.1	Bilan Personnel (GHOULEM)	30
8.2	Bilan Personnel (CHALUMEAUX)	31
8.3	Bilan Personnel (BELUSCA)	32
8.4	Bilan Personnel (DORIATH)	33

Introduction Générale

1.1 Contexte du Projet

Le projet **Fil Rouge** s'inscrit dans le cadre de la troisième année de la filière SRI à l'UPSSITECH et couvre l'ensemble de l'année universitaire. Il est structuré en deux phases, PFR1 et PFR2, visant respectivement le développement du cœur logiciel et du mode manuel, puis l'intégration d'une version autonome du robot.

L'objectif principal est de simuler un environnement de développement professionnel en équipe, mobilisant à la fois les compétences techniques, organisationnelles et de communication. L'encadrement pédagogique joue le rôle de client, validant les choix techniques et fonctionnels tout au long du projet.

Dans ce contexte, l'équipe développe **Upssibot**, un robot capable d'interagir avec l'utilisateur via des commandes vocales ou textuelles, d'effectuer des déplacements dans un environnement simulé et de réaliser une analyse d'image pour détecter et identifier des objets présents dans son environnement.

1.2 Architecture du Projet

L'architecture générale du projet est illustrée dans le diagramme de composants ci-dessous. Elle s'articule autour du module **ControlMod**, qui assure l'interface utilisateur et la gestion des interactions (menus, saisie texte et vocale).

Le traitement des commandes naturelles est réalisé par le module **ManualPilot**, responsable de l'analyse, du parsing et de la génération des actions à exécuter. Les modules **ConfigLoader** et **History** fournissent respectivement la gestion centralisée de la configuration et la journalisation des événements.

La simulation graphique repose sur l'échange entre **SimulatorController** (C) et **Simulator** (Python), permettant l'exécution et la visualisation des commandes. L'architecture présentée se limite volontairement aux modules effectivement implémentés dans le cadre du PFR1.

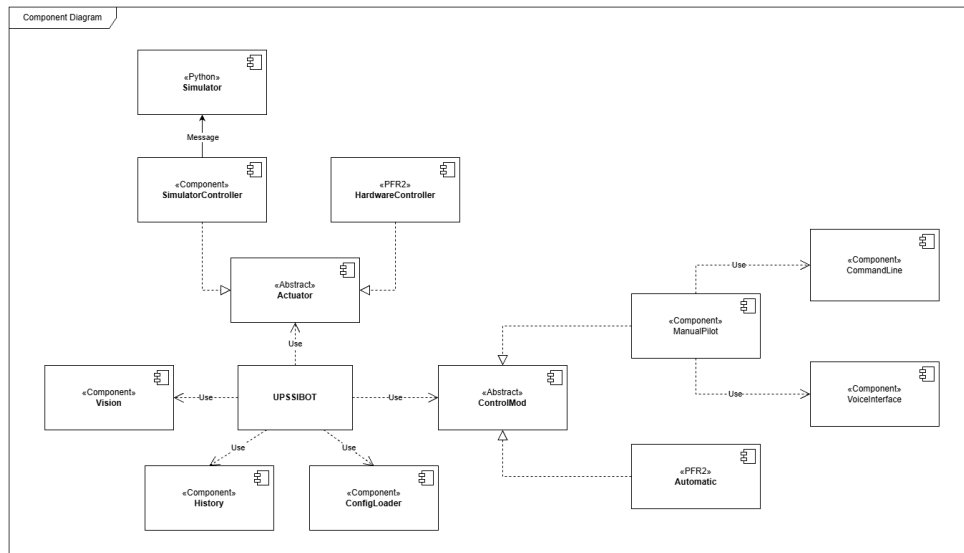


FIGURE 1.1 – Diagramme de composants d’Upssibot

1.3 Objectifs et Répartition

Le PFR1 se concentre sur la conception du cœur logiciel d’Upssibot et sur le développement des modules fonctionnels de base. Le projet inclut le mode manuel permettant l’interaction via commandes textuelles et vocales, la simulation des déplacements et des interactions avec l’environnement grâce au module Python/Turtle, et la mise en place d’une architecture modulaire et extensible.

Le projet vise à assurer une interaction intuitive avec l’utilisateur, permettre le traitement et l’analyse d’images pour détecter des objets et obstacles, garantir la traçabilité et l’historique des commandes et résultats, et développer un code clair et structuré facilitant l’évolution vers la version autonome prévue en PFR2.

Reparation a faire

Pilotage par Commandes (ManualPilot)

2.1 Introduction

L'objectif ici est de partir d'une phrase brute qui va être fournie en sortie de la transcription vocale. Prenons l'exemple suivant :

"Avance de 200 puis tourne à droite"

À partir de là, ma logique est de commencer par découper cette phrase en *tokens* (mots). Ensuite, je vais essayer d'analyser chaque mot individuellement pour pouvoir identifier son type à partir d'un dictionnaire. Ce dictionnaire va contenir, pour chaque commande (**forward**, **backward**, **turn**, **stop**, **direction**, etc.), les synonymes associés.

Dès que le texte d'un token correspond à une commande connue, il va prendre ce type. Une fois que chaque token de notre phrase possède un type qui lui est propre, nous allons avoir des commandes à exécuter que l'on va composer à partir de la disposition et de l'ordre des tokens.

Prenons un cas concret :

Phrase : "Avance de 100 puis tourne de 45"

↓

Résultat : On aura 2 commandes, une pour avancer de 100 et l'autre pour tourner de 45.

Ma logique consiste donc à prendre un verbe d'action et à lui associer la valeur qui vient après, en négligeant tous les tokens inconnus (*unknown*) situés entre les deux. Comme ça, on obtient une liste de commandes propre qui va aller communiquer avec la simulation.

Le programme fournit alors l'action et la valeur pour exécuter, en Python cette fois-ci, les fonctions de la librairie **Turtle** :

- **forward** pour les actions d'avancer,
- **turn** pour les actions de tourner,
- et ainsi de suite, en prenant en compte la valeur associée à l'action.

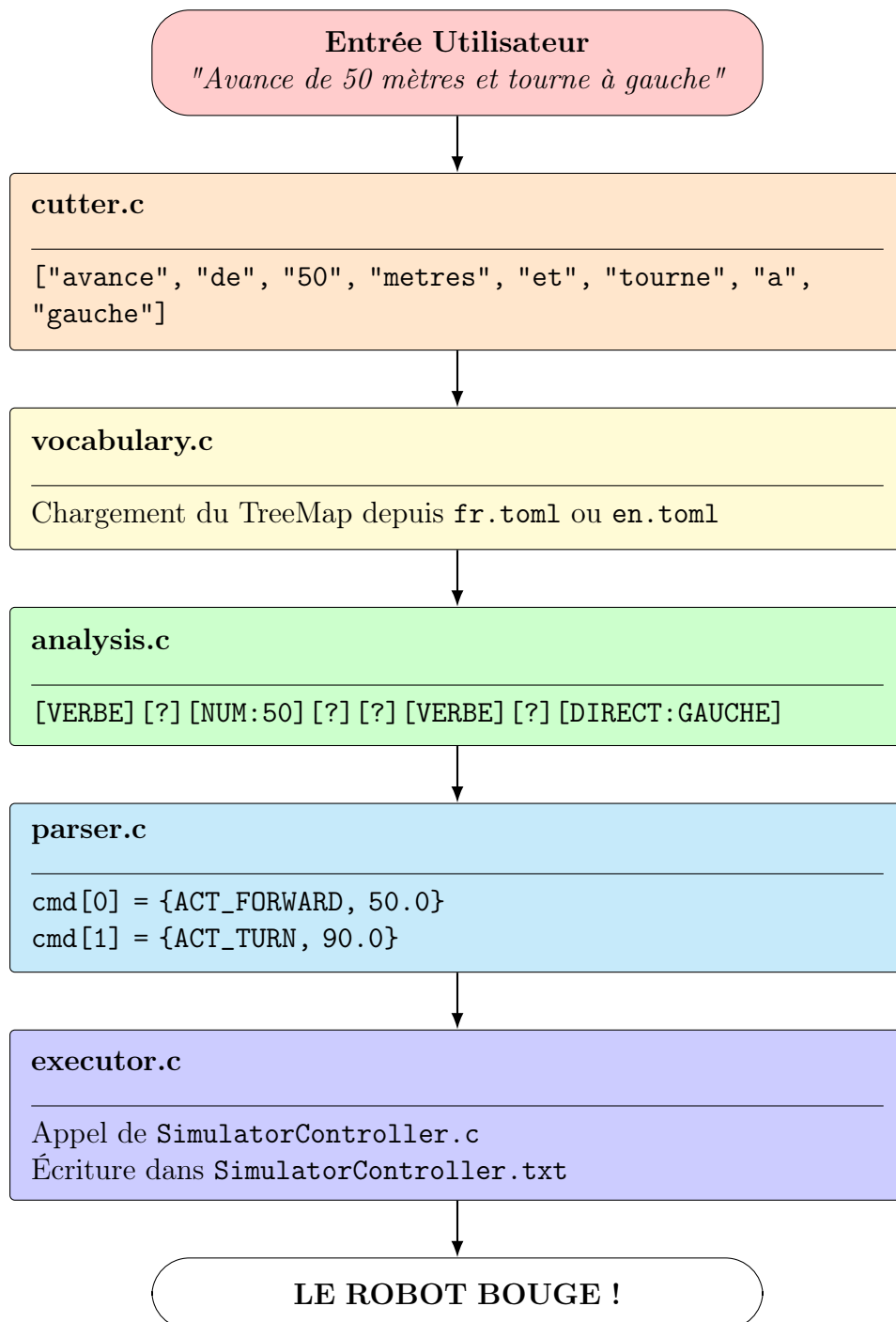
On aura donc le robot qui va bouger en exécutant chaque commande de la liste séquentiellement.

2.1.1 Architecture du ManualPilot

Pour la conception du *ManualPilot*, j'ai voulu avoir une architecture modulaire. On va donc avoir un fichier spécifique pour chaque étape de la pipeline.

Voici le fonctionnement détaillé, fichier par fichier :

- **cutter.h** : Pour commencer, ce fichier est chargé de la découpe de la phrase en tokens (avec un type inconnu pour l'instant) et de mettre tout cela dans une liste. La structure des tokens est définie dans **token.h**.
- **vocabulary.h** : Ensuite, ce fichier va aller charger le fichier de configuration **TOML** (.fr ou .en) qui contient les synonymes de chaque action. Il charge ces données dans un arbre binaire en C géré par **Treemap.h**.
- **analysis.h** : Il va chercher chaque token de la liste dans l'arbre binaire pour récupérer son type. On aura donc en sortie une liste de tokens où chaque élément possède maintenant son type identifié.
- **parser.h** : À partir de cette liste de tokens, on va aller composer des commandes (Verbe + Valeur) dans une liste de commandes, et aussi gérer la négation et les valeurs par défaut. La structure de ces commandes est définie dans **command.h**.
- **executor.h** : Ensuite, on va communiquer avec la simulation. Ce fichier va prendre chaque commande de la liste et appeler les fonctions de **SimulatorController.h**.
- **SimulatorController.h** : C'est lui qui fait le lien final en écrivant l'action et la valeur dans le fichier **SimulatorController.txt**.
- **simulator.py** : Enfin, le script Python va aller lire ce fichier texte et exécuter chaque ligne avec la bibliothèque **Turtle**.



2.2 Étapes Détaillées du Pipeline

2.2.1 Le mot (Token)

Le token (ou mot) est une structure qui va contenir un texte brut, un type identifié (TOK_VERBE, TOK_NUM, TOK_COLOR, TOK_OBJECT, TOK_DIRECTION, TOK_UNKNOWN, TOK_NONE) et enfin une donnée qui peut être soit une action, une valeur, une direction, ou encore une couleur ou un objet (optionnels), j'ai utilisé une **union**, ce qui permet d'avoir une seule de ces valeurs actives à la fois dans la structure :

```

1 typedef struct token
2 {
3     char texte[100];
4     tokentype type;
5
6     /* union : permet d'avoir une seule parmi les valeurs suivantes */
7     union
8     {
9         action_t action;          // pour TOK_VERBE
10        float value;              // pour TOK_NUM
11        color_t color;            // pour TOK_COLOR
12        object_t object;          // pour TOK_OBJECT
13        direction_t direction;    // pour TOK_DIRECTION
14    } data;
15 } token;

```

Listing 2.1 – Définition de la structure Token (token.h)

2.2.2 Séparation en liste de tokens (cutter)

Ce que fait le fichier `cutter.c`, c'est qu'il va commencer par prendre en entrée une liste de tokens vide et une phrase brute. Il va donc copier (avec la fonction `strncpy`) cette phrase dans une variable locale et mettre un caractère nul `\0` à la fin pour être sûr de la sécurité mémoire.

On va ensuite utiliser la fonction `strtok` pour découper la phrase à chaque caractère séparateur suivant (espace, ponctuation, retour à la ligne) : " \t\n,.;:!?'"

On va prendre le token issu de la séparation et on va commencer par le normaliser (mettre en minuscule en ajoutant 32 au code ASCII) avec la fonction `normaliser()`.

Nous allons ensuite rajouter le token dans la liste de tokens qui a été écrite dans `cutter.h` :

```

1 typedef struct tokenlist
2 {
3     token tokenTAB[MAX_TOKENS];
4     int count;
5 } tokenlist;

```

Listing 2.2 – Structure de la liste de tokens

Enfin, on va aussi initialiser le type du token à `TOK_UNKNOWN` avant de passer au prochain mot avec `strtok(NULL, ...)` et incrémenter le compteur de `tokenlist` pour pouvoir le retourner à la fin de la boucle.

2.2.3 Chargement du vocabulaire dans TreeMap (vocabulary)

Ce fichier va charger le fichier `fr.toml` ou `en.toml` (ou autre) en fonction du choix fait par l'utilisateur sur le menu. Ce fichier va contenir des clés et des valeurs (les synonymes).

Par exemple :

```
forward = ["avance", "avancer", "marche", "marcher", "go"]
backward = ["recule", "reculer", "arriere", "back"]
```

On va donc utiliser la fonction `config_loader()` pour lire la valeur dans le TOML et ensuite, grâce à la fonction `add_words()`, traiter chaque action (`forward`, `backward`...) en mettant en paramètre le type et la data :

```
1 phrase = config_loader(filepath, "forward");
2 add_words(&arbre, phrase, TOK_VERBE, ACT_FORWARD, COL_NONE, DIR_NONE);
3 free(phrase);
```

Listing 2.3 – Chargement et ajout des mots au dictionnaire

La fonction va découper la valeur fournie en brut et alors utiliser la fonction `insertValue()` du `TreeMap` pour ajouter chaque token de la liste de synonymes à l'arbre, après avoir spécifié son type et sa data.

2.2.4 Analyse des types (analysis)

Ce fichier va prendre en entrée la liste de tokens (avec le type inconnu), pour ensuite parcourir toute cette liste et, pour chaque token, aller utiliser la fonction `vocabulary_search()` de `treemap.h` pour chercher le token dans l'arbre. Quand celui-ci a été trouvé, on va définir son type.

S'il n'a pas été trouvé, on va utiliser la fonction `is_number()` qui va parcourir chaque caractère. Si jamais le caractère est compris entre '0' et '9' (et qu'il n'y a qu'un seul point pour les décimaux), alors c'est un nombre :

```
1 else if (mot[i] < '0' || mot[i] > '9')
2 {
3     /* Ce n'est pas un chiffre (0-9) */
4     return 0; // pas un nombre
5 }
```

Listing 2.4 – Vérification numérique

Si c'est bien un nombre, on va mettre le type `TOK_NUM`.

Enfin, on a aussi une fonction `print_analysis()` qui nous permet d'afficher la liste pour voir les tokens avec leurs types associés, ce qui est très utile pour le test et la présentation.

2.2.5 La structure de Commande (command.h)

Tout comme pour les tokens, nous avons défini une structure pour les commandes dans `command.h`.

Une commande représente une action associée à des paramètres comme la valeur, la direction, la couleur, l'objet, et aussi la négation qui est un booléen (0 ou 1) :

```
1 typedef struct command
2 {
3     action_t action;
4     float value;
5     color_t color;
6     object_t object;
7     direction_t direction;
8     int negation; /* pour prendre en compte la negation (0 ou 1) */
9 } command;
```

Listing 2.5 – Structure d'une commande

Pour chaque paramètre, le type énuméré correspondant est défini dans `command.h`.

Nous aurons aussi la liste de commandes qui va contenir un tableau de structures (et non de pointeurs pour simplifier la gestion mémoire) ainsi que le compteur :

```
1 typedef struct command_list
2 {
3     command cmd[MAX_COMMAND]; /* Tableau de structures, PAS de pointeurs
4     */
5     int count;
6 } command_list;
```

Listing 2.6 – Liste de commandes

2.2.6 Création de la liste de commandes (Parser)

Cette fonction va prendre la liste de tokens et, après avoir initialisé une liste de commandes vide, on va alors parcourir toute la liste de tokens.

Si on croise un verbe, alors on initialise une commande vide qui va récupérer l'action du verbe avant de passer au token suivant pour vérifier ses données. S'il correspond à un `TOK_NUM`, alors on va remplacer la valeur de la commande :

```
1 /* si c'est un nombre */
2 if (token_list->tokenTAB[j].type == TOK_NUM)
3 {
4     /* remplacer la valeur de la commande par la valeur du token */
5     cmd_list->cmd[cmd_list->count].value = token_list->tokenTAB[j].data.
6     value;
```

Listing 2.7 – Extraction de la valeur numérique

De la même manière, si c'est un `TOK_DIRECTION`, alors on va remplacer la direction de la commande, et si c'est un `TOK_COLOR`, alors on remplace la couleur.

On aura ensuite la gestion de la négation. Au moment où on parcourt la liste de tokens, si jamais on croise un token avec le texte "ne" ou "n" (étant donné que par exemple "n'avance" est séparé par le cutter en "n" et "avance"), alors la variable `negation` passe à 1 (vrai) :

```

1 int negation = 0;
2 if (i > 0 && (strcmp(token_list->tokenTAB[i - 1].texte, "ne") == 0 ||
3               strcmp(token_list->tokenTAB[i - 1].texte, "n") == 0))
4 {
5     negation = 1;
6 }

```

Listing 2.8 – Détection de la négation (avant le verbe)

Et même chose si le token après possède le texte "pas" ou "plus". Donc, si le champ `.negation` de la commande est vrai, on va sauter la commande pour éviter d'avoir une valeur par défaut exécutée par erreur.

On gère aussi le cas où on n'a pas de valeur après le verbe, ce qui va mettre une valeur par défaut (exemple : 50) pour les actions avancer et reculer :

```

1 /* si FORWARD ou BACKWARD sans valeur */
2 if ((cmd_list->cmd[cmd_list->count].action == ACT_FORWARD ||
3     cmd_list->cmd[cmd_list->count].action == ACT_BACKWARD) &&
4     cmd_list->cmd[cmd_list->count].value < 0.001)
5     /* 0.001 pour éviter l'erreur float == 0 */
6 {
7     cmd_list->cmd[cmd_list->count].value = DEFAULT_DISTANCE;
8 }

```

Listing 2.9 – Gestion des valeurs par défaut

On fait la même chose pour les actions de tourner.

On va aussi gérer le cas "demi-tour". Si, lors du parcours de la liste de tokens, on croise le token avec le texte "demi" et le token de l'indice suivant avec le texte "tour", alors on va créer la commande tourner avec une valeur de 180 :

```

1 if (i + 1 < token_list->count &&
2     strcmp(token_list->tokenTAB[i].texte, "demi") == 0 &&
3     strcmp(token_list->tokenTAB[i + 1].texte, "tour") == 0)
4 {
5     init_command(cmd_list);
6     cmd_list->cmd[cmd_list->count].action = ACT_TURN;
7     cmd_list->cmd[cmd_list->count].value = 180.0;
8     cmd_list->count++;
9
10    i += 2; /* on saute les deux tokens "demi" et "tour" */
11    continue;
12 }

```

Listing 2.10 – Cas spécial : Demi-tour

Et pour finir, on gère le cas où l'utilisateur va dire un verbe FORWARD ou BACKWARD suivi d'une direction (mais sans distance). Dans ce cas, on va transformer l'action en TURN car l'intention est de tourner :

```

1 if ((cmd_list->cmd[cmd_list->count].action == ACT_FORWARD ||
2     cmd_list->cmd[cmd_list->count].action == ACT_BACKWARD) &&
3     cmd_list->cmd[cmd_list->count].direction != DIR_NONE)
4 {
5     /* exemple : "avance a droite" devient TURN */
6     cmd_list->cmd[cmd_list->count].action = ACT_TURN;
7     cmd_list->cmd[cmd_list->count].value = 0.0;
8 }

```

Listing 2.11 – Correction contextuelle de l'action

Au final, on aura un tableau `commande_liste[]` prêt à être exécuté.

2.2.7 L'Exécution de la liste de commande (executor)

Cette partie est simple : on prend la liste de commandes et on va parcourir cette liste.

En fonction de l'action de chaque commande (ACT_FORWARD, ACT_BACKWARD, ACT_TURN, ACT_STOP, etc.), on va appeler la fonction correspondante à partir de `SimulatorController.h` (`forward()`, `backward()`, `turn()`) avec un *switch case* et mettre la valeur de la commande en paramètre :

```
1 switch (cmd_list->cmd[i].action)
2 {
3 case ACT_FORWARD:
4     forward(cmd_list->cmd[i].value, robot);
5     history_log(INFO, "Action: ACT_FORWARD executed");
6     break;
7 /* ... autres cas ... */
8 }
```

Listing 2.12 – Dispatch des actions

Il y a une gestion spécifique pour les directions sans valeur explicite : si la direction est DIR_LEFT, on appelle `turn(90)`, et si c'est DIR_RIGHT, on appelle `turn(-90)` :

```
1 else if (cmd_list->cmd[i].direction == DIR_LEFT)
2 {
3     turn(90, robot);
4     history_log(INFO, "Action: ACT_TURN executed to left (90 )");
5 }
```

Listing 2.13 – Gestion des directions par défaut

On va aussi ajouter tout ça à l'historique à chaque fois avec la fonction `history_log()`

2.2.8 Simulation Graphique et Communication (C → Python)

Le fichier `SimulatorController.c` est responsable de la communication entre le langage C et le Python à travers un fichier tampon `SimulatorController.txt`. Il est également chargé de calculer la position théorique du robot dans la fenêtre grâce à la structure définie dans `position.h` :

```
1 typedef struct RobotPosition
2 {
3     float x;
4     float y;
5     float theta;
6 } RobotPosition;
```

Listing 2.14 – Structure RobotPosition

Nous avons donc la fonction `forward()` qui va prendre en paramètre une valeur de distance et permettre de calculer la nouvelle position avec l'équation trigonométrique suivante :

```
1 float new_x = Position->x + distance * cosf(theta_rad);
2 float new_y = Position->y + distance * sinf(theta_rad);
```

Listing 2.15 – Calcul de la nouvelle position

Si la valeur de la distance est négative (< 0), on appelle la fonction `backward()` qui effectue la même logique inversée. Nous avons aussi la fonction `turn()` qui va changer l'angle θ de `RobotPosition`.

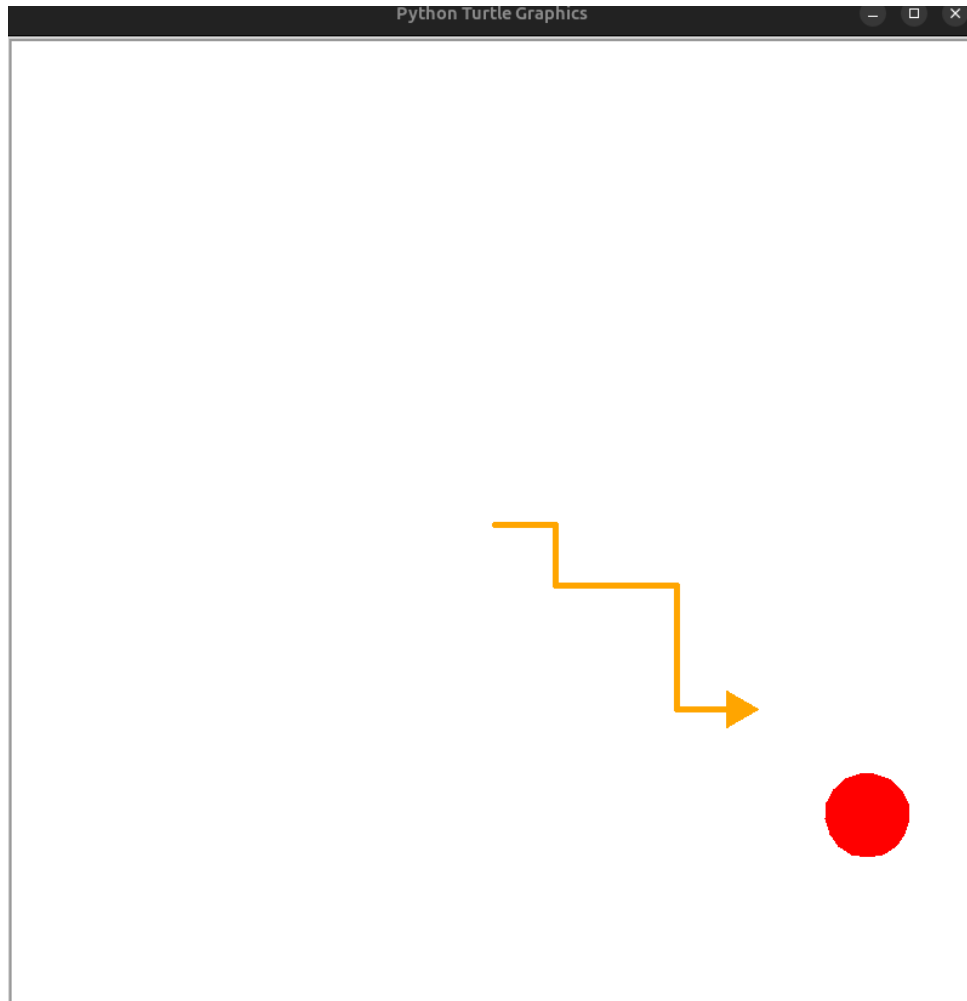


FIGURE 2.1 – Fenetre de simulation

Ces trois fonctions, après avoir modifié la position du robot, vont appeler la fonction `WriteAction()` qui va permettre d'écrire dans `SimulatorController.txt` les actions (`FORWARD`, `BACKWARD`, `TURN`) suivies de la valeur associée.

Le script `simulator.py`, quant à lui, va commencer par ouvrir le fichier et lire les valeurs initiales de x et y pour placer le robot avec `turtle.goto()`. Ensuite, la fonction `ReadAction()` va lire le fichier `SimulatorController.txt` pour récupérer les actions et les valeurs afin d'appeler les fonctions `turtle.forward()` et `turtle.left()`.

De plus, la fonction `ReadScreenConfig()` va lire la hauteur et la largeur de la fenêtre directement à partir du fichier `simulatorConfig.toml` pour pouvoir configurer la taille de la fenêtre dynamiquement.

Pour finir, j'ai rajouté une petite "gamification" pour rendre la simulation amusante : la fonction `draw_ball()` va dessiner une balle rouge qui va avoir une position aléatoire. Si la position du robot après le mouvement correspond à une certaine distance définie dans `check_collision()`, un petit message de victoire s'affiche à l'écran.

Configuration (ConfigLoader)

3.1 Gestion de la configuration

La configuration du projet **Upssibot** repose sur l'utilisation de fichiers au format TOML, permettant de centraliser l'ensemble des paramètres globaux de l'application. Cette approche évite l'utilisation de constantes codées en dur et facilite l'évolution du comportement du programme sans nécessiter de recompilation.

Les paramètres sont chargés dynamiquement au démarrage de l'application par le module **ConfigLoader**, qui fournit une interface simple et générique pour accéder aux valeurs de configuration depuis l'ensemble des modules du projet.

3.1.1 Structure des fichiers de configuration

Structure du `globalConfig.toml`

Ce fichier permet la modification des configurations générales du projet. La section `[language]` définit la langue utilisée par le système, influençant à la fois l'interface utilisateur et le traitement des commandes vocales. La section `[simulation]` permet de configurer le comportement initial de la simulation graphique, notamment l'ouverture automatique de la fenêtre au lancement du programme. Enfin, la section `[path]` centralise les chemins vers les scripts Python externes, incluant le moteur de simulation graphique et les interpréteurs vocaux pour chaque langue. Cette centralisation garantit une séparation claire entre la logique C et les modules Python, tout en facilitant la maintenance et la portabilité du projet.

Structure du `simulatorConfig`

Ce fichier permet de modifier la logique du simulator ainsi que les paramètres de fenêtre turtle. Les sections `[window]` et `[robot]` définissent respectivement les paramètres graphiques de la fenêtre de simulation ainsi que la position initiale du robot dans l'environnement simulé.

3.1.2 Chargement et utilisation des paramètres

L'accès aux paramètres de configuration est assuré par la fonction `config_loader()`, définie dans le module **ConfigLoader**. Cette fonction prend en paramètre le chemin du fichier de configuration ainsi que la clé recherchée, et retourne la valeur correspondante sous forme de chaîne de caractères allouée dynamiquement.

Les valeurs récupérées sont utilisées dans plusieurs modules du projet. Le menu principal s'appuie sur la configuration pour définir la langue active et l'état initial de la simulation. Le module d'interface vocale sélectionne automatiquement le script Python correspondant à la langue configurée, tandis que le module d'historique utilise les chemins définis pour créer et stocker les fichiers de journalisation.

Cette gestion centralisée de la configuration améliore la maintenabilité du code, facilite les tests et prépare l'application à des évolutions futures, notamment dans le cadre du passage à une version autonome prévue en PFR2.

3.2 Implémentation du module ConfigLoader

3.2.1 Construction du chemin et ouverture du fichier

Le chargement d'un fichier de configuration commence par la construction du chemin d'accès complet. Le programme étant exécuté depuis le répertoire de build, un chemin relatif est reconstruit afin de garantir un accès correct aux fichiers situés à la racine du projet.

L'ouverture du fichier est réalisée en mode lecture. En cas d'échec, la fonction retourne NULL, permettant au programme appelant de gérer proprement l'erreur.

3.2.2 Lecture du fichier et récupération de la clé

Une fois le fichier ouvert, celui-ci est parcouru ligne par ligne. Les commentaires et les lignes vides sont ignorés afin de ne conserver que les entrées pertinentes. Chaque ligne valide est analysée pour identifier les couples clé/valeur à l'aide du caractère =.

Lorsque la clé recherchée est trouvée, la valeur associée est copiée dans une zone mémoire allouée dynamiquement, garantissant l'indépendance du module et évitant tout problème lié à la durée de vie des variables locales.

3.2.3 Nettoyage des chaînes

Le nettoyage des chaînes est assuré par la fonction `trim`, qui supprime les caractères blancs en début et en fin de chaîne. Cette étape garantit une interprétation fiable des clés et des valeurs, indépendamment de la mise en forme du fichier TOML.

Cette fonction contribue directement à la robustesse du parsing et à la fiabilité globale du module de configuration.

Interaction Utilisateur et Modes de Contrôle (ControlMod)

4.1 Interface vocale

Afin de proposer une interaction plus naturelle avec le robot, Upssibot intègre une interface de commande vocale. Celle-ci permet à l'utilisateur de piloter le robot à l'aide de phrases prononcées oralement, sans passer par une saisie clavier.

L'interface vocale repose sur une séparation claire entre le langage C et le traitement audio. Le module C `speech.c` agit comme une passerelle entre l'application principale et des scripts Python spécialisés dans la reconnaissance vocale.

4.1.1 Principe de fonctionnement

Le module `speech` fournit une unique fonction, `get_speech()`, chargée de récupérer une commande vocale sous forme de chaîne de caractères. Cette fonction sélectionne dynamiquement le script Python à exécuter en fonction de la langue active, définie dans le fichier de configuration global.

Le chemin du script Python correspondant est récupéré via le module `ConfigLoader`, garantissant une indépendance totale entre le code C et l'implémentation des scripts de reconnaissance vocale.

4.1.2 Exécution du script Python

Une fois le chemin du script déterminé, le module construit une commande système et exécute le script Python à l'aide de la fonction `popen`. Le script est alors chargé de capturer l'entrée microphone, d'effectuer la reconnaissance vocale et d'afficher la commande reconnue sur la sortie standard.

La commande vocale est ensuite retournée sous forme de chaîne de caractères, prête à être injectée dans la chaîne de traitement des commandes (découpage, analyse, parsing et exécution).

4.2 Interface utilisateur et gestion des menus

L'interface utilisateur d'Upssibot repose sur un système de menus textuels implémenté en langage C. Ce choix permet une interaction simple, robuste et indépendante de toute interface graphique, tout en assurant une intégration directe avec les modules de contrôle, de simulation et d'interface vocale.

Le module `menu` constitue le point d'entrée principal de l'application. Il assure la navigation entre les différentes fonctionnalités du système et coordonne les appels aux modules spécialisés, sans réaliser de traitement métier complexe.

4.2.1 Principe général

L'application est structurée autour d'un menu principal donnant accès à plusieurs sous-menus. Chaque menu est implémenté sous la forme d'une fonction dédiée et repose sur une boucle contrôlée par la saisie utilisateur. La sélection des options s'effectue par la lecture d'un caractère depuis l'entrée standard, garantissant une gestion simple et fiable des interactions.

L'affichage de l'interface est généré à l'aide d'instructions `printf`, comme illustré ci-dessous pour le menu principal :

```
1 printf("-----\n");
2 printf("| 1.Control Mode |\n");
3 printf("| 2.Languages   |\n");
4 printf("| 3.Sim Window   |\n");
5 printf("| q.Quit         |\n");
6 printf("-----\n\n");
```

Listing 4.1 – Affichage du menu principal

4.2.2 Fonctionnalités accessibles

Le menu principal permet d'accéder au mode de contrôle du robot (texte ou vocal), à la sélection dynamique de la langue et à la gestion de la fenêtre de simulation. Le menu de contrôle déclenche la chaîne complète de traitement des commandes, depuis la saisie utilisateur jusqu'à l'exécution dans la simulation.

L'ensemble des actions réalisées via les menus est enregistré dans le système d'historique, assurant la traçabilité des interactions et facilitant le débogage.

4.2.3 Rôle architectural

Le module `menu` agit comme une interface entre l'utilisateur et les différents sous-systèmes d'Upssibot. Il centralise la logique de navigation et orchestre les appels aux modules de configuration, d'interface vocale, de pilotage et de simulation, contribuant ainsi à une architecture claire, modulaire et maintenable.

Journalisation et Historique des événements (History)

5.1 Rôle du module d'historique

Le projet Upssibot intègre un système de journalisation permettant de tracer les événements importants survenus durant l'exécution du programme. Ce mécanisme facilite le débogage, l'analyse du comportement du robot et la compréhension globale d'une session utilisateur.

Le module `history` est responsable de la création, de l'écriture et de la fermeture d'un fichier d'historique associé à chaque exécution du programme.

5.2 Niveaux d'événements

Chaque message enregistré dans l'historique est associé à un niveau de gravité. Ces niveaux sont définis à l'aide d'une énumération, permettant de classer les événements selon leur importance.

```
1 typedef enum
2 {
3     INFO ,
4     WARNING ,
5     ERROR
6 } HistoryLevel;
```

Listing 5.1 – Définition des niveaux d'historique

Cette classification permet de distinguer les informations normales de fonctionnement, les situations anormales non bloquantes et les erreurs critiques.

5.3 Initialisation et création du fichier

La fonction `history_init()` est appelée au démarrage de l'application. Elle récupère dynamiquement le chemin du dossier d'historique depuis le fichier de configuration global, puis génère un nom de fichier unique basé sur la date et l'heure de lancement.

Un en-tête est automatiquement écrit dans le fichier afin de marquer le début de la session et de fournir un contexte temporel clair.

5.4 Enregistrement des événements

L'écriture dans l'historique est assurée par la fonction `history_log()`. Chaque appel ajoute une ligne au fichier d'historique contenant l'heure, le niveau de l'événement (`INFO`, `WARNING` ou `ERROR`) ainsi qu'un message descriptif.

Par exemple, lors de la sélection d'une commande par l'utilisateur dans le menu de contrôle, l'action est enregistrée de la manière suivante :

```
1 history_log(INFO, "User Text Input: avancer de 10");
```

Listing 5.2 – Exemple d'utilisation de `history_log()`

Ce mécanisme est utilisé dans l'ensemble de l'application, notamment dans les menus, l'interface vocale, l'exécution des commandes et la gestion de la simulation. Il garantit une traçabilité complète et chronologique des actions utilisateur et des événements système, facilitant le débogage et l'analyse du comportement du robot.

5.5 Fermeture de l'historique

À la fin de l'exécution, la fonction `history_close()` écrit l'heure de fin dans le fichier puis ferme proprement le flux. Cette étape garantit l'intégrité de l'historique et marque explicitement la fin de la session.

Le module d'historique constitue ainsi un support transversal essentiel à la robustesse et à la maintenabilité d'Upssibot.

PARTIE VISION

(DORIATH + BELUSCA)

6.1 Organisation

Le module de vision est séparé en quatre sous modules

- **image.h** : définit ce qu'est une image ainsi que la logique de chargement des fichiers
- **line.h** : définit les notions de lignes, utile dans l'algorithme de reconnaissance de sphère ainsi que pour le futur traitement des cubes
- **color.h** : qui se charge des représentations colorimétriques (RGB ou HSV) ainsi que des calculs de proximité et les extractions.
- **vision.h** : le module public qui définit la logique métier ainsi que les fonctions utiles à l'extérieur du module de vision

6.2 image.h

Il s'agit ici du module qui fait le chargement des images, la lecture des fichiers ainsi que la définition des types de base tel que `Image` et `Pixel`.

Il était important pour moi de ne surtout pas utiliser `scanf` et ses dérivés pour éviter les nombreux risques associés tel que, l'impossibilité de revenir correctement après une erreur, les `buffer overflow`, ainsi que pour respecter les bonnes pratiques modernes. cf [What's wrong with scanf?](#)

d'où l'implémentation plus compliquée que dans d'autres groupes. Cela m'aura permis d'apprendre d'importantes compétences de gestion précise de la mémoire ce qui est particulièrement utile dans le monde professionnel.

6.2.1 line.h

Ce module va être une des briques qui composent l'algorithme de reconnaissance de la sphère ainsi que celui du futur cube. On va alors manipuler des lignes ainsi que des vecteurs 2D qui ont été définis en tant que struct. Ce qui, par exemple pour la sphère, va permettre de trouver son centre.

MathLine est un type qui définit mathématiquement une droite, soit un point de référence et son vecteur directeur.

Vector2d est un type qui définit un vecteur avec ses x et y.

Globalement, voici les différentes fonctions écrites :

```
—1 IntCoordinate findMiddle(IntCoordinate A, IntCoordinate B);  
2
```

Retourne le milieu entre deux points.

```
—1 Vector2d makeRandomVec(void);  
2
```

Permet de créer un vecteur unitaire dans une direction aléatoire qui est utile notamment pour tester la sphère dans différentes conditions.

```
—1 double getVecLength(Vector2d v);  
2
```

Calcule la norme d'un vecteur.

```
—1 MathLine makeLine(IntCoordinate refPoint, Vector2d directingVec);  
2
```

Permet de créer une droite à partir d'un point et d'un vecteur directeur.

```
—1 MathLine perpendicular(MathLine base, const IntCoordinate*  
    crossingPoint);  
2
```

Crée une droite perpendiculaire à une autre. On utilise par défaut le point de référence de la droite "crossingPoint" est donc optionnels.

6.2.2 color.h

On va retrouver plusieurs fonction qui vont vous permettre de passer du système de traitement optique RGB (Rouge Vert Bleu) vers le modèle HSV (Teinte Saturation Valeur). Le choix d'utiliser le HSV a été fait car cela permet d'ignorer la luminosité de l'image et donc un traitement plus simple des pixels.

HSVColor est un type qui définit une couleur au format HSV -> Teinte (H), Saturation (S), Valeur (V).

ColorReferences Une liste de couleurs de référence (couleur que l'on veut détecter)

ColorMasks Les masques d'image, un par couleur (un masque = image noir/blanc indiquant où la couleur est présente)

Globalement, voici les différentes fonctions écrites :

```
1 HSVColor RGBtoHSV(Color base);  
2
```

Convertit une couleur RGB → HSV

```
1 Color HSVtoRGB(HSVColor base);  
2
```

Convertit HSV → RGB

```
1 double colorDistance(HSVColor A, HSVColor B);  
2
```

Mesure à quel point deux couleurs se ressemblent

```
1 ColorMasks extractColors(const Image* img, const ColorReferences  
   colorRef);  
2
```

cherche les couleurs proches des références et crée un masque par couleur.

```
1 void freeColorMasks(ColorMasks* masks);  
2
```

Libère la mémoire des masques.

```
1 bool exportMasks(const char* outFilePath, const ColorMasks masks,  
   const ColorReferences ref);  
2
```

Reconstruit une image de visualisation à partir des masques

6.2.3 détails sur l'isolation des couleurs

Puisque le cas d'utilisation du robot est centré autour de la reconnaissance d'objets de couleur nous avons décidé de directement séparer chaque couleur dans leurs propres matrices binaire que l'on appelle des masques.

Par exemple pour une image quelconque, le masque des pixels rouge contiendra tous les pixels qui ont une plus grande proximité avec la couleur rouge qu'avec toutes les autres couleurs de référence.

La proximité entre les couleurs est définie au sens de la fonction `colorDistance` qui utilise la représentation HSV pour réduire l'importance des variations d'éclairage. La formule utilisée pour la proximité colorimétrique est

$$midS = (S_1 + S_2)/2 \quad (6.1)$$

$$D = \Delta S^2 + \Delta H^2 \times midS \quad (6.2)$$

avec H et S normalisés dans [0.0, 1.0]

cette métrique était initialement basée sur la distance courbe à la surface d'un cylindre unitaire avec H l'angle et S la hauteur, mais nous avons été obligés d'utiliser `midS` comme facteur afin de réduire la dépendance à H lorsque la saturation est réduite. Sans ce facteur nous observions des variations de D qui étaient bien trop chaotiques pour être utilisables.

6.3 vision.h

6.3.1 Détection d'objets

Afin d'améliorer l'efficacité du traitement des images on va chercher à trouver la position de tous les objets potentiels dans une image. Afin de minimiser le travail inutile, cette opération est calculée au besoin sur un masque de couleur en particulier.

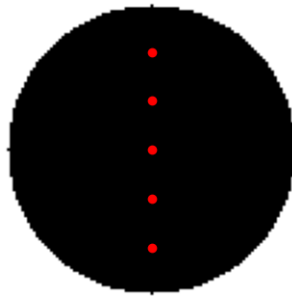
Pour ce faire on utilise une méthode de tag récursif classique. Cependant pour éviter que des reflets et autres artefacts soient considérés comme de multiples objets, on impose une surface minimum (en pixel) pour qu'un cluster de pixels soit reconnu comme un objet.

6.3.2 Reconnaissance de Sphère

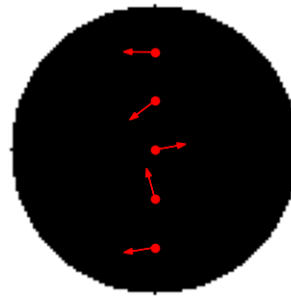
Afin de différencier les sphères des autres objets on a décidé de partir sur un algorithme géométrique. Le principe est de trouver le centre d'un objet, puis d'estimer le rayon d'après le centre et les contours.

Enfin on compare quel quantité de pixels sont hors du contour théorique de l'objet et la quantité de "vide" qui se trouve à l'intérieur du contour. Ces deux grandeurs sont normalisées par la surface de l'objet et nous donnent une signature qui permet de conclure sur la nature de l'objet.

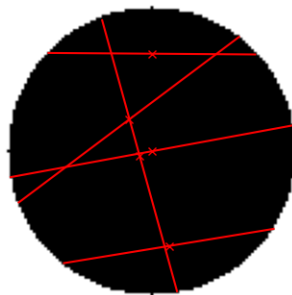
Step 2: 5 Sampled Points Inside the Shape



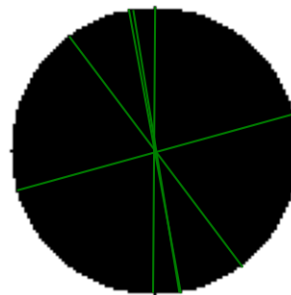
Step 3: Random Probing Vectors



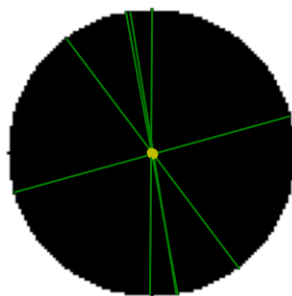
Step 4: Probing Lines and Barycenters



Step 5: Perpendicular Probing from Barycenters



Step 6: Local Center Estimates



Step 7: Final Estimated Circle

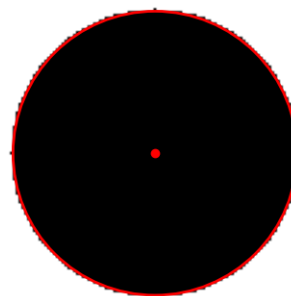


FIGURE 6.1 – Estimation du centre d'une sphère

Puisque l'algorithme est basé sur la géométrie de l'objet lui même, il a l'avantage d'être plutôt résilient face au bruit, et est capable de trouver le centre même avec une image bruitée à 25%

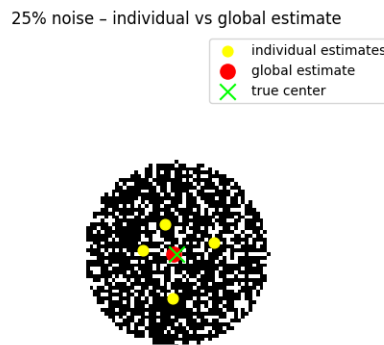


FIGURE 6.2 – Résilience face au bruit

Complexité temporelle

On estime que la complexité temporelle de la construction du modèle (détection du centre et du rayon) est d'environ $\mathcal{O}(nr)$ où n représente le nombre de point échantillonnés et r est la longueur du plus long segment dans l'objet.

Puisque le calcul de la signature nécessite de visiter tout les points de l'objet, on peut en conclure que l'étape de vérification est de complexité $\mathcal{O}(S)$ avec S la surface (en pixels) de l'objet

Cela nous permet donc de conclure que notre processus de décision a une complexité temporelle de l'ordre de $\mathcal{O}(S + nr)$

6.3.3 Résultats

Après une phase d'ajustement pour trouver les hyperparamètres correctes, nous avons observé que notre algorithme était particulièrement efficace pour reconnaître les balles et éviter les cas sans aucune balles.

Cependant, quelques limitations ont été observées, tout d'abord sur certaines images il arrive que l'algorithme donne des résultats différents entre les exécutions. Cela peut être attribué à l'utilisation de vecteur directeur aléatoire qui sont utilisés pour résister aux erreurs de reconnaissance de couleurs, cela reste cependant rare.

Il arrive également que plusieurs balles soient trouvées au même emplacement, ce problème vient probablement de la détection d'objet et devra être corrigé dans le PFR2.

À terme il devrait également être possible (et plus simple) de trouver une meilleure configuration pour les hyperparamètres à l'aide d'un rapide script python et de la librairie scikit-learn.

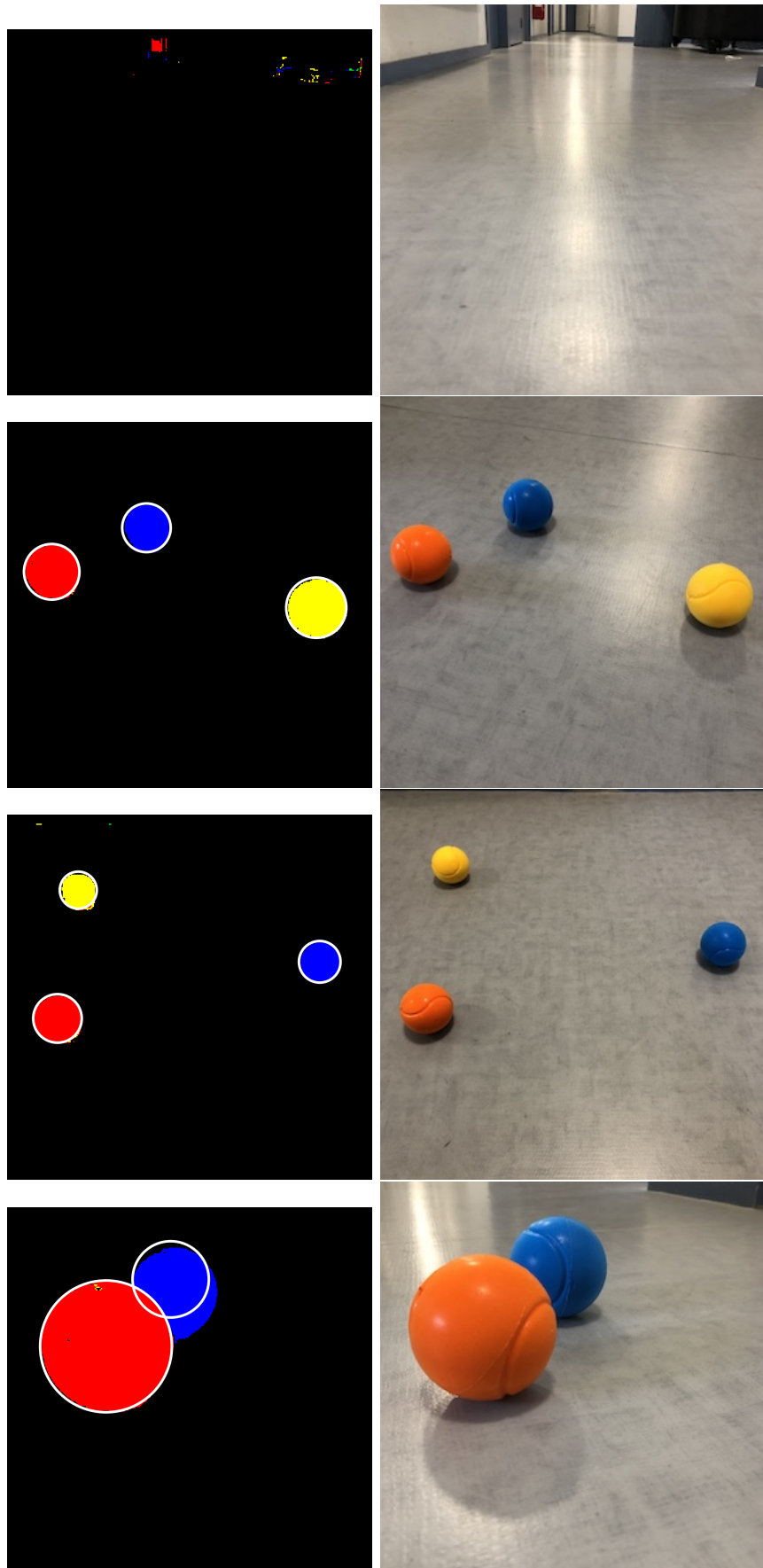


FIGURE 6.3 – Résultat de la détection des balles

6.3.4 Géométrie et réflexion pour le Cube

Avec les étapes réalisées pour la sphère, un schéma s'est créé afin de définir comment nous pourrions détecter, d'une part, que le cube se trouve dans l'image, mais aussi déterminer sa couleur.

On a d'une part récupéré les différentes couleurs de l'image : rouge, vert et bleu. L'idée est alors de travailler sur la sortie qui nous intéresse. Si nous cherchons un cube rouge, alors l'idée aurait été de récupérer la sortie de l'image avec uniquement les pixels rouges dans une matrice, comme précédemment réalisé pour la sphère, et ainsi traiter directement cette matrice de sortie.

La première étape de cette reconnaissance est de trouver les objets contenus dans la matrice (graphe de connexité 8), donc pour cet exemple les objets rouges.

Par la suite voici les différents algorithmes qui devront être réalisés :

- Extraction des contours avec l'algorithme de Canny pour retourner la liste des points du contour.
- Simplification du contour en segments avec l'algorithme de Douglas-Peucker pour retourner une liste de segments de droite.
- Détection des intersections de chaque segment pour retourner la liste des sommets.
- Vérification des angles en les calculant entre chaque segment adjacent.
- Et la vérification des critères composant un cube.

UTILITAIRES (DORIATH)

(Afin de simplifier les tâches de tout le groupe, plusieurs structures de données utilitaires ont été créées)

7.1 matrix.h

Ce module a pour objectif principale de permettre la gestion de donnée en deux dimensions sans pour autant sacrifier la contiguïté de la RAM.

Pour ce faire le tableau a été linéarisé et des fonctions d'accès et d'édition se charge de calculer l'index correct. Un prédicat a également été codé pour permettre de vérifier que les données sur les quels on travaille sont bien valide.

Afin de rendre cette structure générique, le contenu est enregistré sous la forme de `void*`, cela signifie que l'on "oublie" le type de nos données, c'est pourquoi il existe de multiples implémentations des fonctions d'accès et d'opérations standard sur les Matrices qui sont définies dans le fichier `matrix_implementation.h` car malgré la similarité des fonctions, le type diffère et l'absence de méta programming efficace en C rend difficile les structures génériques.

7.2 treeMap.h

Ce module est une implémentation d'un dictionnaire sous la forme d'un arbre binaire à la façon des `std::map` du C++.

Il est principalement utilisé dans le chargement du vocabulaire (cf chapitre 2) car cette structure de donnée permet de lier les mots de l'utilisateur à leur interprétation.

Il était initialement prévu d'utiliser un arbre rouge et noir mais cette partie n'a pas encore été ajoutée et sera probablement implémentée dans le PFR2. Il aurait aussi été possible d'utiliser un arbre radix mais les gains ne justifiaient pas la complexité d'implémentation.

7.3 Vérification qualité

J'ai également réutilisé une github action que je m'étais fabriquée il y a quelques années, celle-ci permet l'analyse automatique du code avec `lizard-analysis` afin de calculer plusieurs métriques de qualité. En particulier le nombre de lignes par fonction, le nombre de paramètres et la complexité cyclomatique.

Cet utilitaire a révélé quelques emplacements dans le code qui risqueraient de devenir compliqués à maintenir à l'avenir. Cela devra être corrigé lors du PFR2. Vous pouvez trouver le rapport de lizart dans le dossier qui vous est fourni.

Bilan

8.1 Bilan Personnel (GHOUILEM)

Durant ce projet, j'ai découvert comment utiliser les bases d'un langage de programmation pour construire une application complète et fonctionnelle. Cela a été l'occasion pour moi de passer de la théorie à la pratique :

- J'ai pu appliquer concrètement des concepts qui étaient théoriques jusqu'ici, comme la manipulation des pointeurs, l'utilisation des structures, et la gestion des tableaux.
- J'ai appris à ne pas tout écrire dans un seul fichier, mais à diviser mon code en plusieurs modules (.c et .h) et à tous les connecter ensemble.
- J'ai beaucoup progressé sur la manipulation de chaînes de caractères en utilisant les fonctions de la bibliothèque `string.h`, et j'ai appris à manipuler des tokens à partir d'une chaîne de caractères pour pouvoir découper et comprendre chaque mot et le sens de la phrase.
- J'ai aussi appris à faire communiquer deux programmes différents en utilisant l'ouverture et l'écriture de fichiers, la lecture ligne par ligne pour récupérer les informations et ignorer tout le reste.
- Ce projet m'a également permis de réviser mes bases en Python pour la lecture de fichiers, et de maîtriser la bibliothèque `turtle` pour visualiser la simulation.
- J'ai aussi appris à travailler en équipe, à avoir une architecture solide pour que tout soit facilement intégrable et que tout fonctionne ensemble, ainsi que la communication avec les autres membres.
- J'ai aussi pris l'habitude d'ajouter de la documentation et des commentaires pour rendre mon code plus propre.
- J'ai aussi appris à utiliser GitHub pour sauvegarder mon travail, gérer mes branches et mes commits, et travailler avec les autres efficacement.
- J'ai aussi appris à travailler avec un client, communiquer, respecter les deadlines, et présenter mon travail.
- J'ai aussi appris à avoir une pensée computationnelle, à schématiser mes idées avant de coder.
- J'ai enfin appris à faire des tests unitaires et à déboguer en utilisant des `printf` et des `fprintf`, ainsi que l'utilisation de Makefile et de Xmake.

En résumé, ce projet m'a permis de gagner en autonomie et de comprendre comment tout assembler pour fournir un livrable de qualité.

8.2 Bilan Personnel (CHALUMEAUX)

Le projet PFR **UPSSIBOT** a été une expérience très enrichissante, me permettant de mettre en pratique les notions vues en cours tout en développant des compétences concrètes en programmation et en organisation de projet.

Mon implication s'est principalement concentrée sur le développement de plusieurs modules importants du projet, notamment la gestion de la configuration, l'interface utilisateur, l'interface vocale ainsi que le système d'historique. J'ai également participé aux choix d'architecture logicielle, avec l'objectif de garder un code clair, structuré et facile à maintenir.

Le travail en langage C m'a permis de mieux comprendre la gestion de la mémoire, la lecture et l'écriture de fichiers ainsi que la gestion des erreurs. L'utilisation de scripts Python pour la reconnaissance vocale a aussi été très formatrice, car elle m'a permis de faire communiquer plusieurs programmes entre eux et d'intégrer des outils externes dans une application en C.

Les principales difficultés rencontrées concernaient la coordination entre les différents modules et le débogage de certaines parties du programme. Ces problèmes m'ont poussé à adopter une méthode de travail plus rigoureuse, notamment en utilisant le système d'historique pour suivre l'exécution du programme et comprendre plus facilement les erreurs.

Ce projet m'a permis de renforcer mes compétences en programmation modulaire, en structuration de projet et en rédaction technique. Il constitue une expérience très utile pour la suite du PFR et représente un travail valorisable dans un portfolio de compétences ou dans un futur contexte professionnel.

8.3 Bilan Personnel (BELUSCA)

La première partie de ce PFR m'a permis de mettre en perspective mes connaissances en informatique dans un domaine appliqué concret. La réalisation de la partie vision était en effet complexe mais m'a permis de réfléchir à des solutions pour pouvoir réaliser la tâche associée.

- Dans un premier temps, j'ai mis en pratique mes connaissances en langage C afin de réaliser un programme permettant l'affichage d'un menu interactif simple intégré au terminal de commande.
- Intégrés et créés des librairies permettant la reconnaissance d'une forme sur une image ainsi que la reconnaissance de la couleur de celle-ci

En bilan, je me suis rendu compte de la difficulté réelle qui était de réaliser ses propres librairies de vision industrielle et des différents domaines dont faisait appel les différentes fonctions. Cependant, mon regret étant de ne pas avoir mis en application mes différentes idées pour la gestion de la reconnaissance du cube car nous nous sommes en premier lieu concentrés sur la partie de reconnaissance de la balle qui nous a pris du temps pour mettre nos différentes idées. De plus, j'ai eu des problèmes avec mes accès à mon compte GitHub, ce qui a eu pour conséquence de ne plus pouvoir faire de push. J'ai donc dû coder avec Live Share, qui permet de coder à plusieurs sur une même machine distante, ce qui a permis de contourner le problème.

Malgré cela j'en retire un excellent exercice très formateur. Ainsi si cela devait être de nouveau réalisé je prendrais autant de plaisir sur ce projet et j'ai hâte de l'intégration sur le robot réel.

8.4 Bilan Personnel (DORIATH)

J'ai apprécié développer le module de vision et en particulier le défis de réaliser mon propre algorithme de reconnaissance d'objet a partir de zéro et je suis plutôt content du résultat.

Bien que ce soit de grand classiques, l'application des structures de données permet de retravailler les bases ce qui est toujours une bonne chose dans le cas des structures standards.

Les contraintes supplémentaires que je me suis ajouté m'ont permis de me dépassé et de retravaillé la gestion de la mémoire a un niveau réaliste, en particulier dans le cas de la lecture de fichier avec résilience aux erreurs utilisateurs. Je regrette cependant qu'il restes quelques `TODO` dans le code, en particulier dans le cas de la gestion d'erreur d'allocation mémoire.

Désormais j'ai hate de passé au PFR2 afin de pouvoir appliqué tout cela sur un véritable robot mobile avec tout les défis temps réel que cela implique.